

Application Requirements Document

Foundation Model Evaluation Platform

Software Engineering Final Project

Omer Aviram

Department of Computer Science
Ben-Gurion University of the Negev

March 23, 2026

Abstract

This document describes the requirements of the software project from the standpoint of the customer.

Currently, evaluating foundation models across various modalities—specifically text and time-series data—requires extensive, repetitive adapter code to align unique model inputs with diverse benchmark formats. The proposed software system solves this integration bottleneck by providing a centralized, lightweight evaluation pipeline.

Through a Python-native web interface, users can select pre-loaded models and benchmarks, configuring evaluation parameters without writing custom scripts. The backend utilizes uniform dataset classes and model wrappers to standardize inputs. It then automatically generates and submits execution scripts to an external compute cluster (e.g., Slurm) to handle the heavy inference workloads asynchronously.

Upon completion, the system parses the raw predictions and provides a visual dashboard for comparing standardized performance metrics (such as MSE and F1-scores). Ultimately, this platform transforms a complex, manual software engineering task into an automated, highly accessible analytical tool, significantly accelerating the research lifecycle.

Contents

1	Introduction	2
1.1	The Problem Domain	2
1.2	Context	2
1.3	Vision	3
1.4	Stakeholders	3
1.5	Software Context	3
2	Usage Scenarios	4
2.1	User Profiles - The Actors	4
2.2	Use-cases	4
2.2.1	Use-Case 1: Execute a Benchmark Evaluation	4
2.2.2	Use-Case 2: View and Export Comparison Dashboard	5
2.3	Special usage considerations	5
3	Functional Requirements	6
3.1	System Initialization & Configuration	6
3.2	User Interaction (Frontend)	6
3.3	Execution Management (Backend)	6
3.4	Evaluation & Scoring	7
3.5	Results Management & Visualization	7
4	Non-functional requirements	8
4.1	Implementation constraints	8
4.2	Platform constraints	8
4.2.1	SE Project constraints	9
4.3	Special restrictions & limitations	9
5	Risk assessment & Plan for the proof of concept	10
5.1	Main Objectives	10
5.2	Technical Risks	10
5.3	Plan for the Proof of Concept	11

Chapter 1

Introduction

This chapter provides an overview of the entire requirements document, providing the "big picture" into which the software project fits.

1.1 The Problem Domain

In the current landscape of foundation model research, evaluating and comparing models is a highly fragmented and labor-intensive process. A major bottleneck stems from the lack of standardization across both evaluation datasets and the models themselves.

First, established benchmarks exist in a wide variety of structural formats. Second, individual foundation models process inputs differently; each model has strict, unique requirements for how different data modalities (such as text versus time-series data) must be formatted and presented.

Consequently, researchers face a difficult integration problem. Running multiple distinct benchmarks on a single model—or evaluating a single benchmark across several different models—requires writing an excessive amount of custom, repetitive adapter code. This lack of a unified evaluation pipeline turns what should be a straightforward analytical comparison into a complex software engineering mission, severely slowing down the research and development lifecycle.

1.2 Context

This software system operates as a centralized, unified evaluation pipeline for foundation models. It sits between the raw benchmark datasets (spanning multiple structural formats) and the foundation models themselves (which handle distinct modalities like text and time-series data).

The platform abstracts away the complex integration layer. The user interacts directly with the platform's interface to configure their evaluation parameters. Behind the scenes, the system utilizes pre-built uniform dataset classes and model wrappers to standardize the inputs and outputs. The processing workload is then delegated to external compute environments, and the results are returned to the platform for aggregation and display.

1.3 Vision

The overall objective of this project is to eliminate the repetitive, custom coding required for model evaluation, thereby accelerating the research lifecycle. By providing a unified platform, researchers will be able to seamlessly run known benchmarks across multiple foundation models—or evaluate a single model across various benchmarks—using a simple, standardized interface. The vision is to transform a complex, multi-step software engineering task into a rapid, accessible analytical process.

1.4 Stakeholders

- **End Users (Researchers & Students):** Individuals who need to run evaluations on multimodal foundation models without writing custom data-loading and formatting scripts from scratch.
- **System Administrators:** Personnel responsible for the computing clusters where the heavy model inference is executed.

1.5 Software Context

The proposed system is an interactive application designed to manage the end-to-end evaluation of foundation models.

- **Inputs:** The user provides evaluation configurations via the interface, selecting specific pre-loaded models, target benchmark datasets, and evaluation parameters.
- **Processing:** The system translates these inputs using uniform dataset classes and model wrappers to ensure compatibility regardless of the chosen modality. It then orchestrates the execution of these evaluations, managing the workflow to the compute environment.
- **Outputs:** Upon completion, the system processes the raw predictions, calculates standardized evaluation metrics, and outputs the results to a visually accessible dashboard for easy comparison.

Chapter 2

Usage Scenarios

In this chapter, the main usage scenarios of the software system are described.

2.1 User Profiles - The Actors

Actors are entities that are external to the system and exchange data with it. This system interacts with two primary actors:

- **Standard User (Human):** A researcher or student evaluating foundation models. This user has a basic understanding of machine learning concepts (e.g., prompting, time-series forecasting, performance metrics) but wants to avoid writing custom adapter code. They interact with the system via a graphical web interface to configure evaluations and view results.
- **Compute Cluster (Non-Human):** The external computing environment (e.g., a university Slurm cluster) responsible for executing the heavy model inference. It receives batch job scripts from the platform, executes the model evaluations on GPU resources, and writes the output logs back to a shared file system where the platform can retrieve them.

2.2 Use-cases

2.2.1 Use-Case 1: Execute a Benchmark Evaluation

- **Actor:** Standard User, Compute Cluster
- **Pre-condition:** The user has selected a valid pre-loaded foundation model and a target benchmark dataset from the interface.
- **Post-condition:** The evaluation completes, and the raw performance metrics are saved to the system’s local database.
- **Process Flow:**
 1. The user configures the evaluation parameters (e.g., selecting a time-series benchmark and a specific foundation model) and clicks “Run”.

2. The system formats the benchmark using the uniform dataset class and wraps the model to handle the specified modality.
 3. The system generates an execution script (e.g., an `sbatch` script) and submits it to the external Compute Cluster.
 4. The UI updates to show the user that the job is "In Progress".
 5. The Compute Cluster finishes the evaluation and outputs the results.
 6. The system parses the results and saves the metrics (e.g., F1, MSE) to the database.
- **Alternative Scenario:** The Compute Cluster lacks available resources. The system informs the user that the job is queued and will execute when resources are freed.

2.2.2 Use-Case 2: View and Export Comparison Dashboard

- **Actor:** Standard User
- **Pre-condition:** At least one evaluation run has been successfully completed and logged in the system.
- **Post-condition:** The user views a visual comparison and exports a CSV file of the results.
- **Process Flow:**
 1. The user navigates to the "Dashboard" tab within the platform.
 2. The system queries the local database for all completed benchmark evaluations.
 3. The system generates visual charts (e.g., bar charts comparing MSE across different models for a specific time-series dataset).
 4. The user clicks the "Export Results" button.
 5. The system compiles the displayed metrics into a CSV file and prompts the user's browser to download it.

2.3 Special usage considerations

Because foundation models require substantial GPU memory and compute power, evaluations cannot be strictly real-time. The system must handle asynchronous processing gracefully. Users must be aware that submitting an evaluation places the job into a compute queue, and wait times will vary based on external cluster availability. The platform's UI must reflect this by clearly separating the configuration phase from the asynchronous results-viewing phase.

Chapter 3

Functional Requirements

This chapter describes the major functions to be handled by the software, organized by logical system components.

3.1 System Initialization & Configuration

- **Load Supported Models:** The system shall load the configurations for pre-wrapped foundation models upon startup, distinguishing between text-only and multimodal (text and time-series) capabilities.
- **Load Benchmark Datasets:** The system shall instantiate the uniform dataset classes, making the supported benchmark datasets available for user selection without requiring structural modifications.

3.2 User Interaction (Frontend)

- **Evaluation Configuration:** The system shall provide an interactive interface allowing the user to select one or multiple foundation models and one or multiple benchmark datasets.
- **Parameter Tuning:** The system shall allow the user to input specific evaluation parameters, such as context window limits or few-shot prompt configurations for in-context learning tasks.
- **Execution Trigger:** The system shall provide a mechanism (e.g., a "Run Evaluation" button) to initiate the pipeline based on the selected configuration.

3.3 Execution Management (Backend)

- **Job Script Generation:** Upon triggering an evaluation, the system shall automatically generate a batch job script (e.g., an `sbatch` script) containing the necessary environment variables, paths, and execution commands.
- **Job Submission:** The system shall submit the generated job script to the external compute cluster's queue manager.

- **Status Tracking:** The system shall monitor the execution status of submitted jobs (e.g., Queued, Running, Completed, Failed) and reflect this current status within the user interface.

3.4 Evaluation & Scoring

- **Uniform Data Processing:** The system shall format the selected benchmark data using the predefined uniform dataset classes before passing the data to the model wrappers.
- **Metric Calculation:** Upon job completion, the system shall parse the raw model predictions and calculate standardized metrics based on the data modality (e.g., exact match or F1-score for text tasks; Mean Squared Error (MSE) or Mean Absolute Error (MAE) for time-series forecasting).

3.5 Results Management & Visualization

- **Data Storage:** The system shall save the calculated evaluation metrics, along with the run metadata (model name, benchmark name, timestamp, execution time), into a local database or structured file system.
- **Dashboard Display:** The system shall query the stored results and display a visual comparison (e.g., summary tables or bar charts) of model performances across the selected benchmarks.
- **Data Export:** The system shall allow the user to export the aggregated evaluation results as a downloadable CSV report for external analysis.

Chapter 4

Non-functional requirements

This chapter describes constraints that affect the way the software system is specified, designed, implemented, and tested.

4.1 Implementation constraints

- **Performance:** The web interface must remain responsive (e.g., screen transitions and standard queries loading in under 2 seconds) even while heavy evaluation jobs are running in the background. Because foundation model inference is computationally expensive, actual job execution time is bounded by the external hardware and queue status, not the platform itself.
- **Safety & Security:** The proof-of-concept will operate in a single-user local environment. However, the architectural design assumes that any future production deployment must implement authenticated user sessions to prevent unauthorized access to expensive GPU compute resources.
- **Usability:** The system assumes users have a baseline understanding of machine learning evaluation metrics (e.g., F1, MSE) and prompting techniques. No advanced programming knowledge should be required to run an evaluation.
- **Reliability & Stability:** The system must gracefully handle external cluster failures (e.g., node crashes or queue timeouts). Instead of crashing the application, the system shall catch the error logs and display a clear, user-friendly failure status on the dashboard.

4.2 Platform constraints

- **Development Stack:** The platform must be developed primarily in Python to seamlessly integrate with the existing research codebase (dataset classes and model wrappers). UI frameworks will be limited to Python-native libraries (e.g., Streamlit or Gradio).
- **Compute Environment:** The backend execution relies heavily on the availability of a Slurm-managed GPU cluster for evaluating large foundation models.

4.2.1 SE Project constraints

Developing this software within the context of a university course imposes specific presentation constraints:

- **Demonstration Limitations:** Live execution of multimodal foundation models on large benchmark datasets can take hours and depends entirely on cluster availability. Therefore, for the final project demonstration, the system must support an "offline" or "simulated" mode.
- **Simulation Strategy:** The live demo will rely on either a drastically reduced "dummy" dataset for live inference, or pre-computed benchmark logs that the system can parse and visualize instantly. This will demonstrate the data processing, workflow automation, and UI capabilities without waiting in a cluster queue.

4.3 Special restrictions & limitations

- **Data Immutability:** The system must process standard benchmark datasets in their original structural formats using uniform data classes. It cannot require the raw benchmark files to be manually altered or pre-processed prior to evaluation.

Chapter 5

Risk assessment & Plan for the proof of concept

In this chapter, we outline the plan for developing a proof-of-concept (POC) prototype.

5.1 Main Objectives

The primary objective of this POC is to determine the viability of integrating a lightweight user interface with an external compute cluster for automated model evaluation. It is designed to test our implementation options for the uniform dataset classes and reduce the overall risk of project failure.

Specifically, the planned prototype is expected to provide:

- **Better understanding of requirements:** Clarifying exactly what inputs are needed from the user to successfully trigger a complete evaluation pipeline.
- **Understanding of technology limitations:** Testing the latency and reliability of submitting batch jobs (e.g., via Slurm) from a Python-based web framework.
- **Early assessment of design decisions:** Validating the architecture of the uniform dataset wrappers to ensure they can handle both text and multimodal time-series formats without requiring manual code changes per run.
- **Visualization for the customer:** Allowing reviewers to see the look-and-feel of the dashboard and how evaluation metrics will be compared.

5.2 Technical Risks

- **Cluster Integration:** The application risks freezing or timing out if the external compute queue is heavily loaded.
- **Data Parsing:** Different foundation models require highly specific prompt configurations and context windows. A risk remains that the uniform data wrapper might fail to properly format edge-case benchmarks.

5.3 Plan for the Proof of Concept

To address these risks and prove the system’s core functionality, the POC will implement a limited, vertical slice of the application:

1. **Basic UI Setup:** Develop a single-page interface (using a native Python library) containing a dropdown to select one pre-loaded foundation model and one standard time-series benchmark.
2. **Mock Evaluation Execution:** Implement a "Run" button that takes the UI selection, wraps the dataset, and successfully generates an execution script. For the POC, this script will either execute a highly truncated "dummy" evaluation on the local machine or retrieve a pre-computed output log to simulate cluster execution.
3. **Metric Parsing & Display:** Prove that the system can read the raw output of this mock evaluation, calculate a standard metric (e.g., MSE), and display it on the UI in a simple table or chart.

By successfully executing this pipeline, the POC will demonstrate that the foundational architecture—from user input to data wrapping to results visualization—is sound and ready to be expanded with additional models and actual cluster deployment.